E-Commerce Price Tracker (PricePulse)

Full System Documentation, Architecture, Execution Log & Codebase

1. System Architecture Overview

PricePulse is a production-grade automated e-commerce price monitoring and time-series analytics engine. It employs a polyglot persistence architecture combining **SQL relational storage** (SQLite via SQLAlchemy ORM) for structured product records and price history checkpoints with **NoSQL document storage** (MongoDB) for unstructured raw HTML snapshots and HTTP metadata payloads.

Layer / Subsystem	Primary Technologies	Core Responsibilities
Relational DB	SQLite / SQLAlchemy ORM	Categories, products, price observations, scrape execution logs
Document Storage	MongoDB / PyMongo	Raw HTML snapshots, DOM payloads, request headers
Scraper Core	Requests / BeautifulSoup4	HTTP session pooling, custom user-agent headers, DOM extraction
Analytics Engine	Pandas / NumPy	Moving averages, standard deviation, percentiles, buy/wait signals
Visualization	Matplotlib (Agg)	Headless Base64 PNG trendline charts and target price threshold lines
Web & Automation	Flask / Jinja2 / APScheduler	Interactive UI, REST endpoints, CSV export, non-blocking cron jobs

2. Troubleshooting & Error Resolution Matrix

Error Encountered	Root Cause	Resolution Implemented
AttributeError: SQL_DATABASE_URI	Config key mismatch with DATABASE_URL	Implemented fallback aliasing in sql_database.py
ModuleNotFoundError: apscheduler	Package not installed in virtualenv	Executed pip install apscheduler inside active venv
AttributeError: SCRAPE_INTERVAL_HOURS	Missing default in config class	Updated config.py to load default 6-hour interval
SyntaxError in scheduler.py	Typo during terminal file redirection	Rebuilt clean single-line import in scheduler.py
Browser Bing Search Loop	Pasted markdown brackets into address bar	Navigated directly to pure localhost:5000 URL

3. Verification Test Logs & End-to-End Status

```
============================= test session starts ==============================
rootdir: C:\Users\pasal\E-Commerce-price-Tracker
collected 22 items

tests/test_database.py ......................... [ 22%]
tests/test_mongo.py ............................ [ 45%]
tests/test_analytics.py ........................ [ 68%]
tests/test_services.py::test_add_product_to_track PASSED
tests/test_services.py::test_get_product_details_with_analytics PASSED
tests/test_visualization.py::test_price_history_chart_success PASSED
tests/test_app.py::test_dashboard_route PASSED
tests/test_app.py::test_add_page_route PASSED

======================== 22 passed in 10.47s ========================
```

4. Core Service Controller (app.py)

```
from flask import Flask, render_template, request, redirect, url_for, flash, jsonify, Response
from config import Config
from database.sql_database import SQLDatabase
from database.mongo_database import MongoDatabase
from scraper.product_scraper import ProductScraper
from services.price_tracker import PriceTrackerService
from services.scheduler import PriceTrackerScheduler

app = Flask(__name__)
app.config.from_object(Config)

sql_db = SQLDatabase()
sql_db.init_db()
mongo_db = MongoDatabase()
scraper = ProductScraper()
service = PriceTrackerService(sql_db=sql_db, mongo_db=mongo_db, scraper=scraper)

# Start background scheduler
scheduler = PriceTrackerScheduler(service=service)
scheduler.start()

@app.route('/')
def index():
    products = sql_db.get_all_products(active_only=False)
    return render_template('index.html', products=products)
```

5. Docker & Multi-Container Setup

```
services:
  mongodb:
    image: mongo:7.0
    ports: ['27017:27017']
    volumes: [mongo_data:/data/db]
  web:
    build: .
    ports: ['5000:5000']
    environment:
      - DATABASE_URL=sqlite:///data/ecommerce.db
      - MONGO_URI=mongodb://mongodb:27017/
```